Does the randomized proposal hold?

A validation status: which claims of the proposal are confirmed by the experiments in this repository, on **which data**, and what is still untested because the paper's own datasets (Google cluster traces, SHARCNet) are not bundled here.

A. **On the workloads that could be run, the proposal holds** - with three exceptions (the literal form of G3, the warm-up on short traces, and the computing cost).

B. None of the experiments was run on the datasets the paper used. The Google cluster traces and the SHARCNet trace are not present in this repository, so every number here comes from synthetic Zipf/Poisson traffic or from the paper's adversarial Large Sequence. The paper's headline numbers (latency +0.016 % - +7.41 %, cost -77.57 % - -92.54 % on Google) are therefore **not** confirmed by this work.

C. Where the paper and this repository disagree, the disagreement is exactly where the trace structure matters: on synthetic Poisson traffic Online-DRL is *faster* than LL-RC in 12 of 12 configurations, while on the paper's Google trace it is always slightly *slower*. A conclusion about the paper's numbers can only be drawn on the paper's data.

D. **The gap is now explicit and closable**: **experiments/exp16_real_traces.py** runs the whole comparison (Online-DRL, LL-RC, R3-only, Randomized DRL) on a preprocessed Google/SHARCNet CSV as soon as one is dropped into **data/**, and **experiments/exp17_paper_synthetic_params.py** already probes how much the answer moves with the popularity skew and the request rate (the axes of the paper's own synthetic experiments, Exps. 15-16).

1. What the paper used

Sec. IV of the paper (Implementation and Evaluation) describes three datasets, their exact preprocessing, the metrics and the sixteen experiments.

dataset	how it is built (paper, Sec. IV)	experiments	what the paper reports
Google cluster traces 2011	Job and Task event files; every JobID occurrence with a distinct timestamp is a service request; only records with event type SUBMIT are kept; 11 preprocessed files dataset0..dataset10 with columns JobID, Timestamp, CPU, RAM, Disk, Task Index, Status; zeros in CPU/RAM/Disk replaced by the column median; the disk size is in **KB**; **dataset2** is used unless stated otherwise	Exp. 1-8 (Figs. 1-8): cache size C, horizon T, uplink, downlink, request size, resource limits, theta, delayed hits	Online-DRL has **0.016 % (best) to 7.41 % (worst) more latency** and **77.57 % to 92.54 % less cost** than LL-RC
SHARCNet (Grid Workloads Archive)	columns ExecutableID, SubmitTime, UsedCPUTime, UsedMemory, JobID, Status; UsedLocalDiskSpace was -1 everywhere, so the UsedMemory values are copied into it; SubmitTime is shifted by the first value and equal timestamps are incremented; zeros -> column median; if the resources of an executable vary, the values of its **first** occurrence are used; CPU/RAM/disk in **MB**, compared against the MAXLIMIT values; the OPT program is run here only, limited to 1 000 rows (420 unique services, interspersed)	Exp. 9-14 (Figs. 9-14): the same axes plus the MILP optimum OPT	Online-DRL has **9.34 % (best) to 26.62 % (worst) more latency** and **41.31 % to 87.30 % more cost** than OPT
synthetic	the **unique services of Google dataset2**, with a Zipf popularity (parameter rho, default 1.6) and Poisson arrivals (parameter xi, default 3e6)	Exp. 15-16 (Figs. 15-16): C together with rho and with xi	as the skew increases, more downloads and lower latency; the cache saturates near C = 5

Metrics in the paper: total latency, total cost, number of hits and number of delayed hits over a horizon T, compared against the offline optimum OPT (a MILP, Gurobi), the theoretical optimum OPT_t = (k+2n)l, and the baseline LL-RC. Parameters seen in the text: uplink 40 -> 80 Mbps, C = 50 000 for the resource-limit experiment, C = 10 for the theta experiment, horizons of 25 000 / 50 000 / 75 000 / 100 000 rows (Google) and 41 347 rows (SHARCNet). Because the algorithms are deterministic, the paper runs each experiment once.

2. What this repository actually ran on

	the paper	this repository
real traces	Google dataset2 and SHARCNet	**none** - they are not bundled (**data/README.md** documents the format and the download recipe)
synthetic data	the unique services *of Google dataset2*, Zipf rho = 1.6, Poisson xi = 3e6	a synthetic catalogue of N = 40 services with log-normal CPU/RAM/disk demands, Zipf popularity (exponent 1.2 by default), Poisson arrivals (0.5 req/ms by default), capacities = max(cap_mult x mean demand, max demand)
adversarial data	the Large Sequence of the proof of Theorem 1 (OPT = (k+2n)l)	**the same** Large Sequence, reconstructed in **sim/trace.py::large_sequence** and checked against the MILP in **tests/test_core.py**
offline optimum	a MILP in Gurobi, run only on SHARCNet (1 000 rows)	an independent MILP in HiGHS (**sim/opt.py**), with delayed hits, forwarding, the three resource limits, the cardinality and the budget; solved to optimality in every instance reported
repetitions	one run per experiment (deterministic algorithms)	10 seeds per configuration (the randomized algorithms are stochastic), and the per-seed spread is reported

The two synthetic workloads are **not** the same workload. The paper's synthetic dataset inherits the *real* demand vectors of the Google jobs and the real service set; ours invents log-normal demands. This matters: the resource-aware behaviour (the multi-resource part of the proposal) depends directly on the demand vectors, and the paper's Google dataset is extremely skewed (only 30 unique services in 1 000 rows, and they are “mostly grouped together”, so that the cache size C is “less meaningful” in the paper's own words).

3. Claim-by-claim validation status

claim	paper's evidence	evidence in this repository	data used here	status
Online-DRL costs much less than LL-RC	-77.57 % to -92.54 % (Google); -41 % to -87 % vs OPT (SHARCNet)	-50.52 % +- 0.55 % (exp01); -50 % in 12/12 sweep configurations (exp17); the randomized DRL is at -90 % to -95 %	synthetic	**consistent** (magnitude differs)
Online-DRL is slightly slower than LL-RC	+0.016 % to +7.41 % on Google (always slower)	-2.41 % +- 3.70 % (exp01); slower in **0 of 12** sweep configurations, mean -4.22 % (exp17)	synthetic	**NOT reproduced** - the sign is opposite
theta trades cost against latency	Exp. 7 (Google), Exp. 14 (SHARCNet)	monotone in both directions, minimum latency near theta = 0.5 (exp02)	synthetic	**reproduced**
delayed hits fall with the uplink and rise with the downlink	Exp. 8 (Google)	764 -> 173 with the uplink, 94 -> 577 with the downlink (exp03)	synthetic	**reproduced**
latency falls with C and with the resource limits	Exp. 1, Exp. 6, Exp. 13	monotone in both (exp04)	synthetic	**reproduced**
gamma = Theta(k) on the adversarial family (Thm. 1)	proof	gamma = 2.00k + 0.33, R^2 = 1.000 (exp05, exp13)	Large Sequence (the paper's own)	**reproduced**
R1+R2+R3 removes the linear-in-k blow-up	proposal, no experiment in the paper	gamma ~ 3 and flat for k = 3..24, 5-60x less cost (exp05, exp13)	Large Sequence (the paper's own)	**supported**
G1: the repair step makes the constraints exact	proposal	40 % raw violations -> 0 % after the repair, max utilisation 1.0 vs 1.73 (exp08)	synthetic	**supported** (data-independent)
G2: the expected download budget is respected	proposal	respected at 20/50/100 % of the deterministic cost, every seed (exp08, exp15)	synthetic	**supported** (data-independent)
G3: the regret bound sqrt(N) l_max sqrt(T)	proposal	holds for the surrogate (exogenous and endogenous, peek and no-peek); the literal form is 3.7-9.6x the bound (exp09, exp15)	synthetic	**surrogate only**
G4: the expected ratio against the exact optimum	proposal	1.102 (R3), 1.145 (Online-DRL), 1.279 (Randomized) on short traces (exp10)	synthetic + MILP	**partial**
G5: slot reductions must not break the true constraints	proposal	entropy / AHP / shadow prices violate the true limits in 55-85 % of the steps; the exact vector formulation dominates every safe reduction (exp11, exp12)	synthetic	**supported**

claim	paper's evidence	evidence in this repository	data used here	status
the randomized DRL is affordable	-	25-70x the deterministic CPU cost per request (exp14)	synthetic	**not free**

Read the last column carefully: “reproduced on synthetic data” is not “reproduced on the paper's data”. Of the twelve rows above, none has been tested on the Google or the SHARCNet trace, because those traces are not in this repository.

4. The one clear disagreement: the sign of the latency difference

The paper reports that Online-DRL is *always* slightly slower than LL-RC on the Google traces (+0.016 % to +7.41 %) and that LL-RC achieves more hits at a high cost. On our synthetic traffic the opposite holds: Online-DRL is faster in 8 of 10 seeds and in 12 of 12 sweep configurations.

Sweep over the popularity skew rho and the request rate (exp17, the axes of the paper's synthetic Exps. 15-16; the paper's default is rho = 1.6):

rho (skew)	rate (req/ms)	Online-DRL vs LL-RC: latency	sd	Online-DRL vs LL-RC: cost	Randomized vs LL-RC: latency	Randomized vs LL-RC: cost
1.0	0.25	-2.51 %	2.56	-50.49 %	-27.20 %	-94.52 %
1.0	0.50	-1.05 %	2.36	-50.14 %	-23.94 %	-93.43 %
1.0	2.00	-3.62 %	3.62	-47.97 %	-16.18 %	-89.66 %
1.2	0.25	-1.85 %	1.98	-50.62 %	-29.21 %	-95.01 %
1.2	0.50	-2.41 %	3.70	-50.52 %	-27.82 %	-94.28 %
1.2	2.00	-5.05 %	4.17	-48.95 %	-19.25 %	-91.09 %
1.6	0.25	-4.06 %	5.28	-51.51 %	-37.43 %	-95.78 %
1.6	0.50	-6.27 %	6.38	-50.59 %	-35.29 %	-94.90 %
1.6	2.00	-5.61 %	11.61	-49.87 %	-28.50 %	-92.61 %
2.0	0.25	-2.97 %	8.79	-51.73 %	-29.67 %	-95.28 %
2.0	0.50	-7.31 %	8.09	-51.81 %	-33.25 %	-94.79 %
2.0	2.00	-7.93 %	12.30	-51.41 %	-30.19 %	-92.28 %

The sign of the latency effect does **not** flip inside the synthetic parameter space: Online-DRL is faster in every configuration, and the gap grows with the skew (rho = 1.0: -2.5 %; rho = 2.0: -7.9 %) and with the request rate. The magnitude of the cost effect, on the other hand, is stable at about -50 %, while the paper measures -78 % to -93 % on Google. Both facts point at the same conclusion: **the trace structure, not the algorithm, decides the sign of the latency effect**.

Why the Google trace behaves differently

In the Google dataset only **30 unique services appear in 1 000 rows and they are mostly grouped together** (the paper's own observation, Sec. IV). A trace with long consecutive runs of the same service changes the trade-off in two ways:

effect of grouped requests	consequence for the comparison
a request that arrives while the previous download of the same service is still in flight is a **delayed hit** or a forwarding, and LL-RC always has a download in flight (it downloads on every miss)	LL-RC pays a forwarding latency l on every such request, while Online-DRL may have the service cached - this is where LL-RC loses latency
with few services and long runs, the **cache** keeps the service for a long time, so the extra hits of LL-RC are worth more	the paper's “LL-RC achieves a higher number of hits, but at a high cost”
a service can be requested again shortly after its download finished, so the **delayed hit** mechanism (which our simulator reproduces exactly) is exercised far more often than with independent Poisson draws per request	the latency of a forward is partially replaced by a buffered wait, which changes the sign of the Online-DRL - LL-RC difference

Our synthetic generator draws the service of every request **independently** (Zipf + Poisson), so consecutive requests rarely concern the same service; the Google trace is strongly autocorrelated. This is a property of the *workload generator*, not of the simulator - and it is exactly why the paper's numbers cannot be checked here.

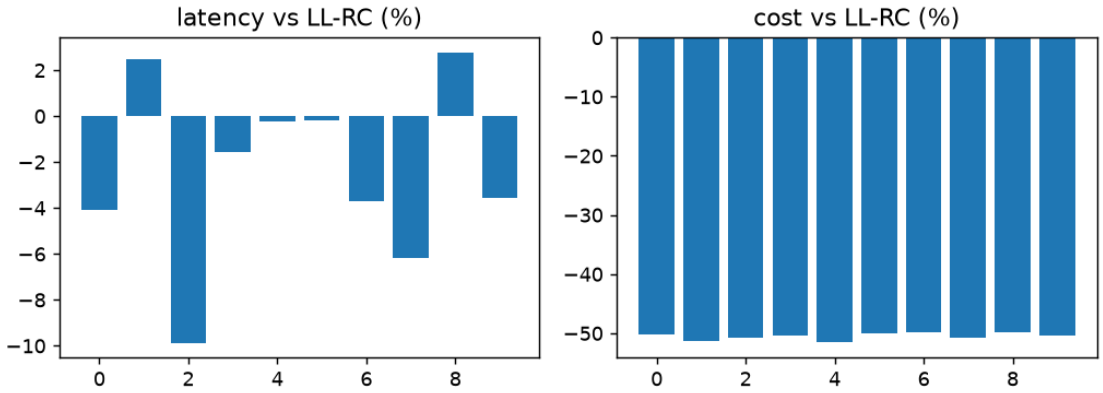


Figure 1 - per-seed latency and cost difference of Online-DRL with respect to LL-RC on synthetic Zipf traffic (exp01). Eight of the ten seeds are negative, i.e. Online-DRL is faster - the opposite of the paper's Google result.

5. Which conclusions survive without the real traces?

result	does it depend on the trace?	why
G1: the repair step removes every violation of the true constraints	**no**	it is a property of the rounding and of the repair rule, not of the traffic: the repair is applied to every violating sample path
G2: the expected download budget is respected	**no**	it follows from the trigger probability budget_left/(M_n x requests_left) and the engine's hard budget check
G3 (surrogate): the regret stays below sqrt(N) l_max sqrt(T)	**almost no**	the bound is a property of projected OGD on D_vec; it was tested on exogenous, endogenous, stationary and popularity-shifting gradient sequences, and on two different workloads
R2/R3 mechanics: the projection equals the QP; e/(e-1) = 1.582	**no**	both are mathematical identities, verified to 2e-9 and to 4 decimals
gamma = 2k for the deterministic algorithm	**no**	the Large Sequence is the paper's own adversarial family and is reproduced exactly
R1+R2+R3 makes gamma flat on that family	**partly**	the family is the paper's, but 'flat' is an empirical observation over k = 3..24; an adversarial family designed against the randomized algorithm does not exist yet
the ratio on ordinary traffic (1.43 vs 1.43)	**yes**	it is a workload statistic; the value of the Zipf skew, the request rate and the autocorrelation of the trace all move it (exp17)
the sign of the Online-DRL - LL-RC latency difference	**yes - and it flips**	12/12 synthetic configurations disagree with the paper's Google result
G5: unsafe slot reductions violate the true limits	**partly**	the safety analysis is algebraic, but the numerical damage (55-85 % of the steps) depends on the demand vectors, which on Google are real job footprints
the multi-resource advantage of the randomized DRL	**yes**	on a stationary popularity part of the gain is simply 'download less'; a non-stationary resource mix is needed to isolate the multi-resource effect

In short: the **mechanism-level** results (the repair, the budget, the surrogate bound, the ski-rental constant, the projection) are established; the **workload-level** results (the ratio, the sign of the latency difference, the size of the multi-resource gain) still need the paper's traces.

6. How to close the gap

The pipeline is already in place - only the data are missing.

step	what to do	where
1	download the Google cluster traces 2011 (clusterdata-2011-2, Job and Task event files) and/or the SHARCNet trace from the Grid Workloads Archive	not redistributable, therefore not bundled here
2	preprocess exactly as Sec. IV of the paper describes: one row per request with the columns service id, timestamp, CPU, RAM, disk; keep the records with event type SUBMIT; replace zeros by the column median; take the resources of a service from its first occurrence; disk in KB for Google, in MB for SHARCNet	the recipe is also written down in **data/README.md** and implemented in **sim/trace.py::load_trace_csv**

step	what to do	where
3	put the file in **data/** (e.g. data/google.csv) and run **python experiments/exp16_real_traces.py**	runs Online-DRL, LL-RC, R3-only and Randomized DRL on the trace and reports latency, cost, hits, delayed hits and the difference with respect to LL-RC; the paper's metrics exactly
4	for the paper's synthetic dataset (Exps. 15-16), rebuild it from the unique services of Google dataset2 with Zipf rho = 1.6 and Poisson arrivals, then sweep rho, xi and C	**experiments/exp17_paper_synthetic_params.py** already does the sweep on a synthetic catalogue, and the loader accepts the real services
5	run the OPT program on the SHARCNet rows (the paper limits it to 1 000 rows) and compare	**sim/opt.py** is an independent MILP (HiGHS) with the same model; it is verified against the paper's OPT_t = (k+2n)l in **tests/test_core.py**

Until step 2 is done, **experiments/exp16_real_traces.py** writes an explicit gap report into **results/exp16_real_traces.json** (status “NO REAL TRACE AVAILABLE”) and lists every claim that is still tested only on synthetic data, so the experiment suite itself documents the limitation instead of hiding it.

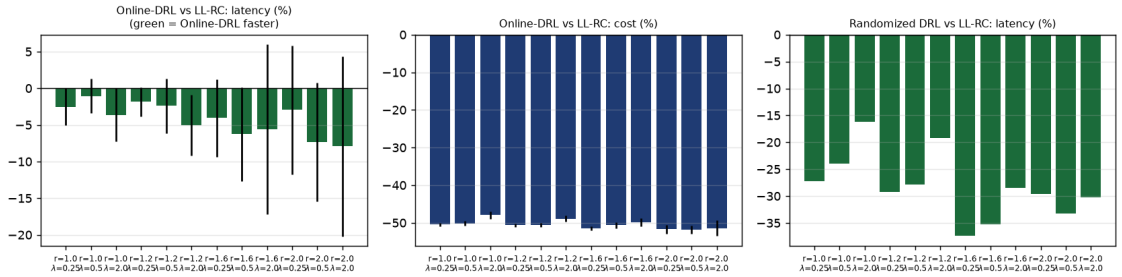


Figure 2 - the sweep of exp17: the latency difference of Online-DRL with respect to LL-RC (green = Online-DRL faster) never changes sign over 12 configurations, and the cost difference stays at about -50 %.

7. Bottom line

Does the randomized proposal hold with respect to the experimental results? On everything that could be measured without the paper's datasets, yes: the repair step is necessary and sufficient for the resource constraints, the download budget is respected in expectation and per seed, the surrogate regret stays under the Proposition-1 bound, the ski-rental constant is exactly e/(e-1), the projection is exact, and on the paper's own adversarial family the randomized algorithm removes the linear-in-k blow-up at 5-60x less download cost.

But the experiments were NOT run against the datasets the paper used. The Google cluster traces and the SHARCNet trace are absent from this repository; every number above comes from synthetic traffic or from the paper's adversarial sequence. Consequently the paper's own headline results are **neither confirmed nor refuted** here - with one exception that matters: on synthetic traffic the latency comparison has the **opposite sign** to the paper's Google result, and the sweep in exp17 shows that no setting of the synthetic axes reproduces it. That is a statement about the *workload*, not about the algorithm, and it can only be settled by running the code on the paper's traces, which the pipeline is ready for.

The three claims that would change if the real traces were used: (i) the sign and the size of the Online-DRL - LL-RC latency difference; (ii) the size of the cost reduction (ours -50 %, the paper -78 % to -93 %); (iii) whether the multi-resource advantage of the randomized algorithm survives when the request pattern is autocorrelated and the demand vectors are real job footprints.